Now, we can construct the estimate of the CDF for the symbols using the following:

```
> CDFsbyTicker = lapply(cancelbyticker, kCDF, bw="SJ")
```

I want to evaluate each of the inverse CDFs at a regular grid of probabilities ranging from 0.1 to 0.9:

```
> pseq = seq(0.1,0.9, by=0.1)
>
> holdquantiles = matrix(ncol=length(pseq),
+                        nrow=length(CDFsbyTicker))
> for(i in 1:length(CDFsbyTicker))
+ {
+    holdquantiles[i,] = CDFsbyTicker[[i]]$invCDF(pseq)
+ }
```



**Exercise:** What just happened in this previous code? What was the point of this?

Converted all of the sample distributions of the number of cancellations into a "standard form"

Eg. You believe/propose that properties of cancellation dist. are predictive of future properties of a stock

Could summarize dist. using mean, SD

Here, using a richer source of info, the percentiles

The smoothing serves to reduce the variance in the estimates of these percentiles

## Smoothing Relationships

Smoothing techniques are also very commonly applied to situations where one variable is naturally thought of as a response, i.e., a quantity to be predicted, and other variables are considered as predictors.

The discussion that follows presents an overview of nonparametric regression. The focus is placed on constructing useful summaries that will lead to better understanding of the relationship between the response and the predictor(s).



## Example: Pricing Options

A European call option on a stock is a contract that gives the holder the right to purchase that stock at the named strike price on the expiration date.

An American call option is the same, but allows the holder to purchase the stock at that price **on or before** the expiration date.

The classic Black-Scholes Theory for option pricing establishes a price for a European option as a function of the following: the strike price, the time to expiration, the current asset price, the volatility of the price, and the current risk-free rate of return.

We will consider a data set consisting of 1,000 call options sampled on September 29, 2017.

To generate this sample, NYSE stock symbols were randomly chosen, and then one currently-listed call option was randomly chosen for each equity, weighted by the volume of trading.

Information recorded for each were as follows: The strike price, the time to expiration, the current asset price, the historical volatility (based on daily returns of 30 most recent trading days), the implied volatility and the Black-Scholes price for the option (with a risk-free rate of zero assumed).



The full R code to generate such a sample can be found on Canvas. There are some important components worth pointing out:

The package `quantmod` includes a function `getOptionChain()` which allows one to easily obtain the current options information for a ticker symbol.

The package `fOptions` includes `GBSOption()` and `GBSVolatility()`, which can be used to calculate the Black-Scholes price and the implied volatility for an option, respectively.

The package `TTR` includes a function `stockSymbols()` which will return all of the ticker symbols (along with other useful information) for the AMEX, NASDAQ, or NYSE exchange.

```
sampleoneoption = function(symbol, opttype, daysvol)
{
  require(fOptions)
  require(quantmod)
  require(dplyr)

# Sample a call option

   holdout = getOptionChain(symbol,NULL)

   alloftype = unlist(holdout, recursive=FALSE)
   alloftype = alloftype[seq(1,length(alloftype),by=2) +
                              as.numeric(opttype=="put")]
   alloftype = bind_rows(alloftype,.id="date")

# Choose one at random, weighted by volume
   touse = sample(1:nrow(alloftype),size=1,prob=alloftype$Vol)
   hold3 = alloftype[touse,]


# Calculate time to expiry

   holdstr = hold3$date
   expiry = as.Date(holdstr,format="%b.%d.%Y")
   timetoexpiry = as.numeric(expiry - Sys.Date())


# Calculate the historical volatility

   histdat = getSymbols(symbol, auto.assign=FALSE, from = Sys.Date() -
daysvol)

   logrets = dailyReturn(Ad(histdat))[-1]
   histvol = sd(logrets)*sqrt(252)


# Find most recent price

   curprice = as.numeric(last(Ad(histdat)))


# Find Implied volatility

   implvol = try(GBSVolatility(hold3[,6],substring(opttype,1,1),
             curprice,hold3[,2],timetoexpiry/365,0,0),TRUE)

   if("try-error" %in% class(implvol))
   {
      implvol = NA
   }
```

```
# Find BS prediction

   bsval = try(GBSOption(substring(opttype,1,1),curprice,
            hold3[,2],timetoexpiry/365,0,0,histvol)@price,TRUE)

   if("try-error" %in% class(bsval))
   {
      bsval = NA
   }


# Output Results

   list(timetoexpiry = timetoexpiry,
   strike = hold3[,2],
   last = hold3[,3],
   bid = hold3[,5],
   ask = hold3[,6],
   volume = hold3[,7],
   openint = hold3[,8],
   curprice = curprice,
   histvol = histvol,
   implvol = implvol,
   bsval = bsval)
}

#setwd("~/Teaching/MSCF/DScourses/LectureNotes/Part7_Smoothing/
OptionsData")

library(quantmod)
library(fOptions)
library(TTR)
library(dplyr)

allNYSEstocks = stockSymbols("NYSE",quiet=TRUE)
allNYSEsymbols = allNYSEstocks$Symbol

# Number of days of data to use to calculate historical volatility
daysvol = 30

# How many options do we want?
target = 1000

# Check to see if some already done

if(length(dir(pattern="optionsample.Robj")) > 0)
{
   load("optionsample.Robj")
```

```
   numgoodones = nrow(optionsample)
   set.seed(numgoodones)
   print(paste("Restarting search. Already found",numgoodones))
} else
{
   optionsample = NULL
   numgoodones = 0
   set.seed(0)
}

datout = NULL
symbs = NULL


while(numgoodones < target)
{
   cand = sample(allNYSEsymbols, size=1)
   print(cand)

# Check to see if there is any option info for this symbol

   holdout = try(getOptionChain(cand,NULL,auto.assign=F),TRUE)

   if("try-error" %in% class(holdout) || length(holdout) == 0)
   {
      next
   }


# Check to see if there is enough daily market data
# on this symbol.

   holdout = try(getSymbols(cand,auto.assign=F),TRUE)

   if("try-error" %in% class(holdout))
   {
      next
   }

   if(nrow(holdout) < daysvol)
   {
      next
   }


# This symbol seems to be fine

   print(cand)

   holdopt = sampleoneoption(cand, "call", daysvol)
```

```
   if(numgoodones == 0)
   {
      optionsample =
data.frame(symbol=cand,holdopt,stringsAsFactors=FALSE)
   } else
   {
      optionsample = rbind(optionsample,c(symbol=cand,holdopt))
   }

   save(optionsample,file="optionsample.Robj")

   numgoodones = numgoodones + 1
   set.seed(numgoodones)
}
```

**Exercise:** The following appears as part of the provided code.

```
> holdout = try(getOptionChain(cand,NULL,auto.assign=F),TRUE)
>
> if("try-error" %in% class(holdout) || length(holdout) == 0)
+ {
+     next
+ }
```

What is the purpose of this syntax?

This portion returns "0" if the command returned an error or if there were no options listed. If occurs, move on and sample another option.



This sample can be found in the Data Sets area on Canvas, with the name `optionssample09302017.txt`.

```
> optionsdata = read.table("optionssample09302017.txt",
+                          sep=",", header=T)
```

Our objective is to explore the relationships between the current ask price for the option and the available variables.

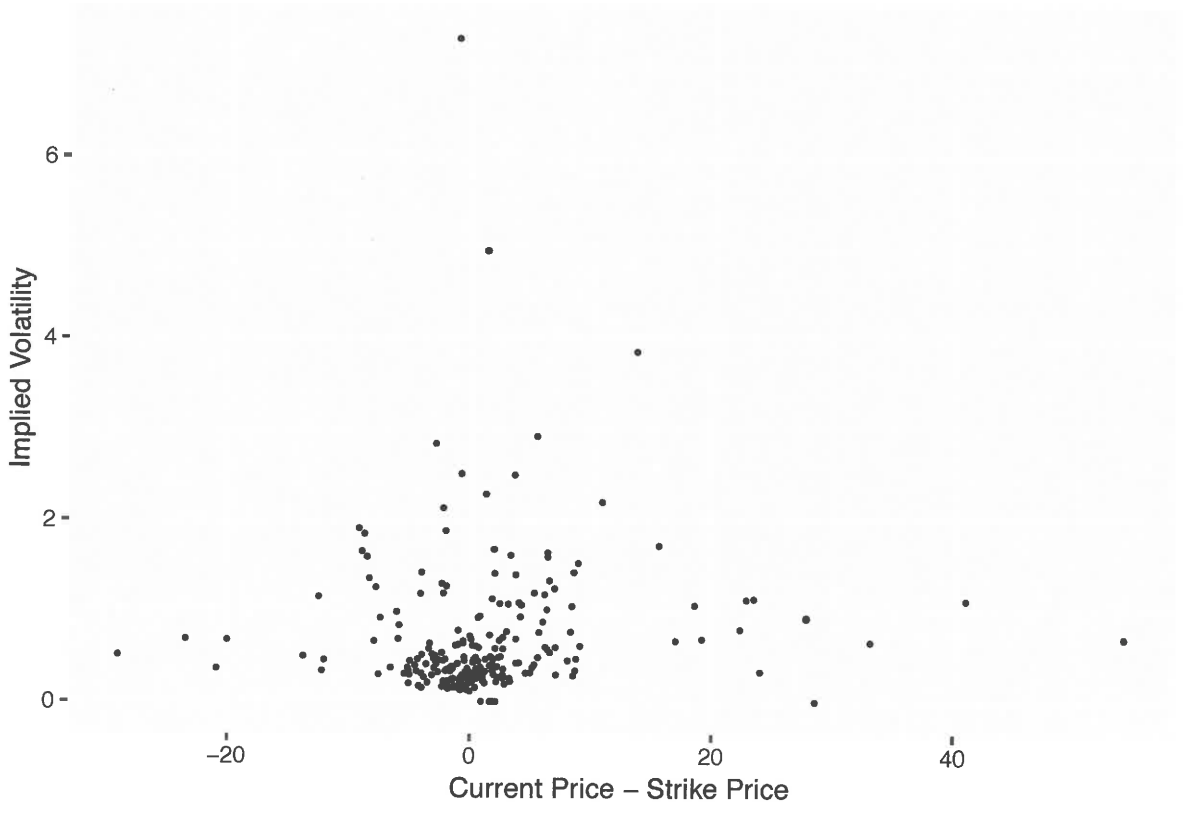
This is an "empirical" alternative to Black-Scholes theory, whose limitations are well-known, mainly due to the failure of the underlying normality assumption.

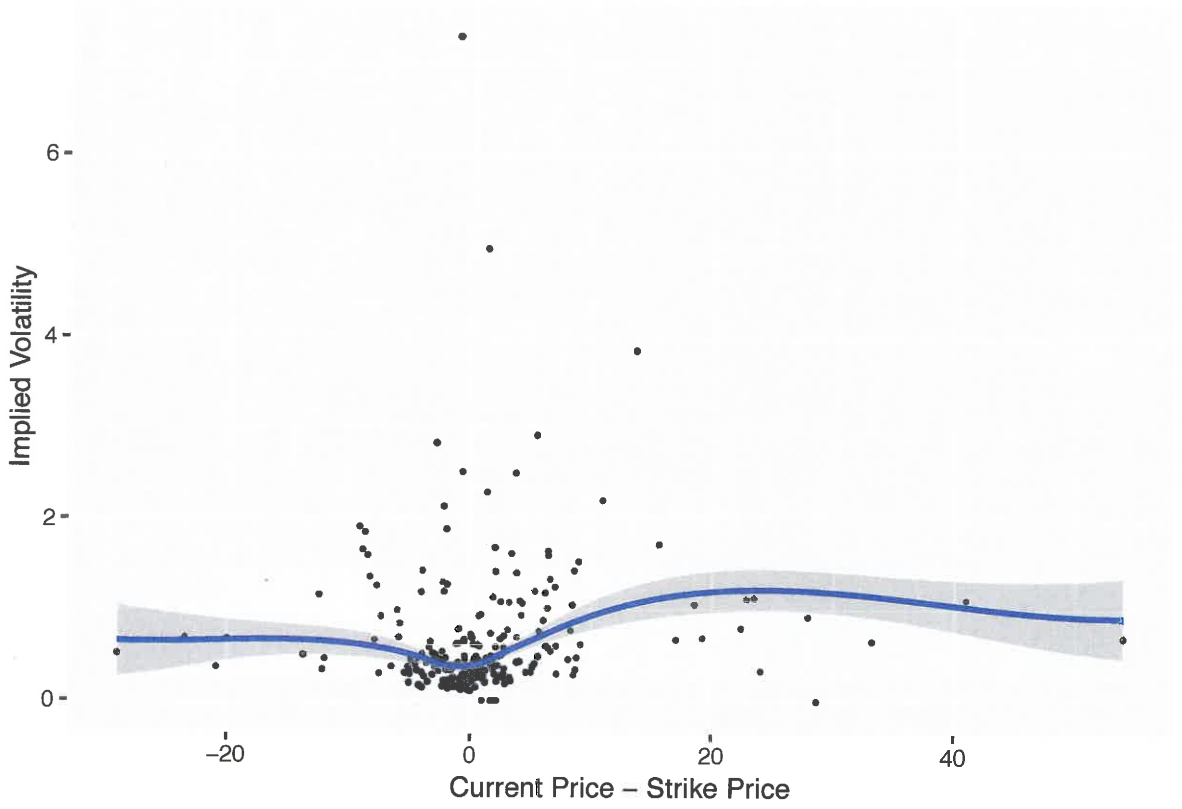
A classic manifestation of the failure of Black-Scholes is the so-called volatility smile. Under the theory, a plot of implied volatility versus how far "in the money" the option currently is should be flat for options with the same expiration date.

Let's create this plot in our case. Is there evidence of the smile here?

```
> optionsdata$inmoney = optionsdata$curprice -
+                       optionsdata$strike
> optionsdata20 = filter(optionsdata, timetoexpiry==20)
> ggplot(optionsdata20, aes(x=inmoney,y=implvol)) +
+   geom_point(size=0.5) +
+   labs(x="Current Price - Strike Price",
+        y="Implied Volatility")
```










## Local Linear Regression

The figure on the previous page shows a nonparametric regression or nonparametric smooth of the scatter plot.

There are many variants of nonparametric regression, but here we focus on local linear regression, a version of local polynomial regression. This approach has proven to be very powerful, and possesses many strong theoretical properties to justify its use.

Briefly stated, this procedure works by fitting a sequence of linear models: Each is fit not to the entire data set, but to only data within a neighborhood of a target point. The size of this neighborhood is a smoothing parameter: Large neighborhoods yield a large degree of smoothing, while small neighborhoods result in minimal smoothing.

Our model here is that we observe $(x_i, Y_i)$ for $i = 1, 2, \ldots, n$ and that

$$Y_i = f(x_i) + \epsilon_i$$

not assuming $f(x) = \beta_0 + \beta_1 x$, e.s.

where the $\epsilon_i$ are iid with mean zero and variance $\sigma^2$. Assuming that the $\epsilon_i$ are normal will lead to further nice properties, but this development does not require that assumption.

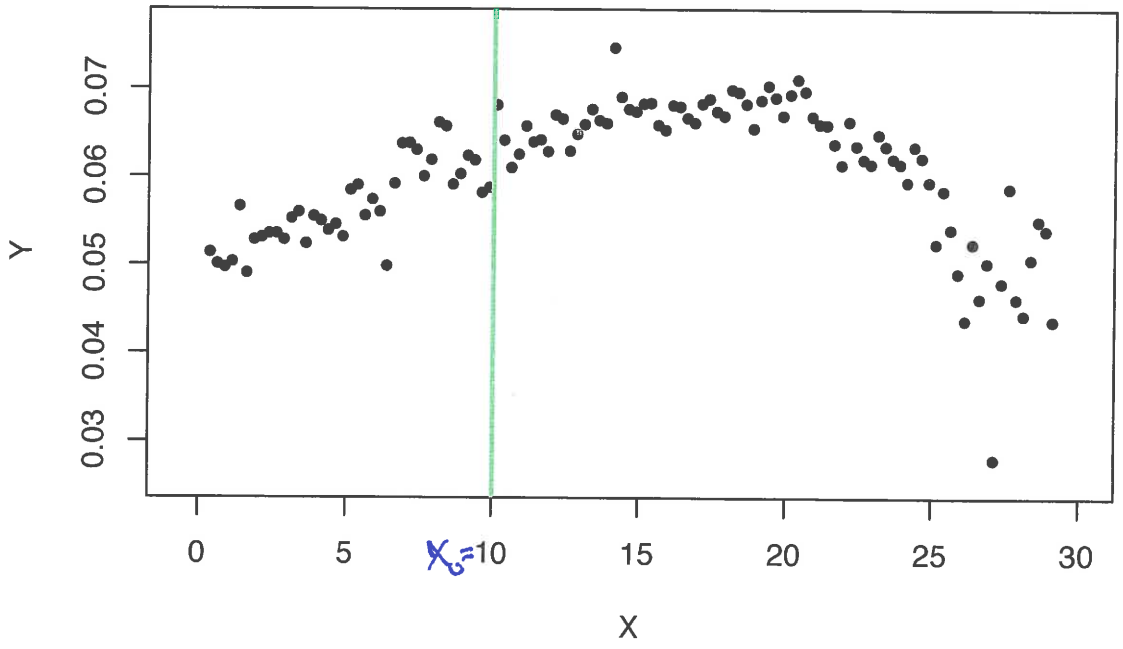
In order to construct the local linear regression estimate of $f(\cdot)$, it is best to consider a sequence of steps for each fixed $x_0$ at which $f(\cdot)$ will be estimated.

(For context on these data, see Ruppert and Matteson, Section 11.3.)



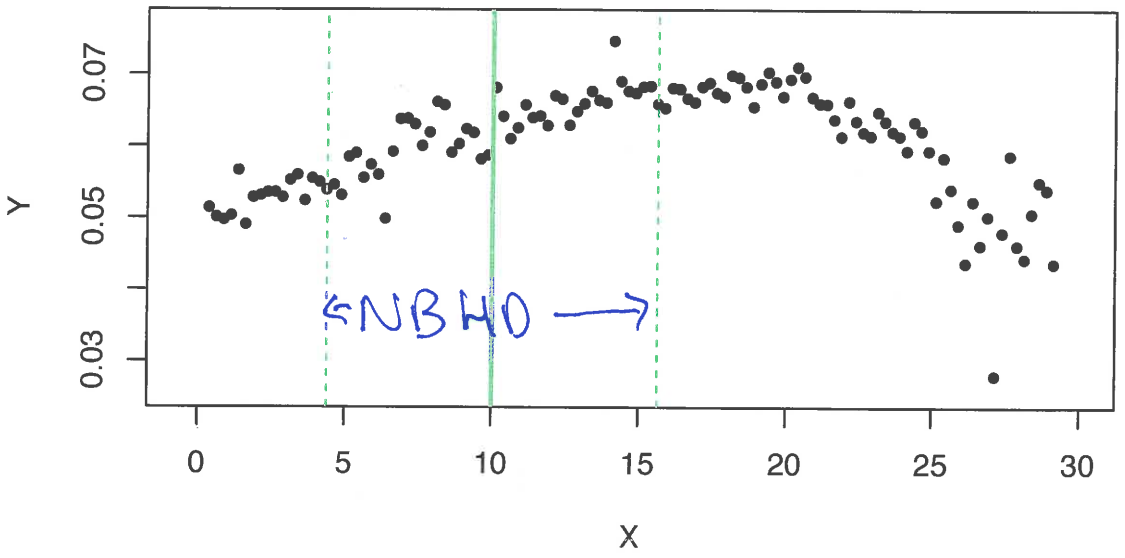
**Step One: Fix the target point $x_0$.**

Our objective is to estimate the regression function at $x_0$. $f(10)$



**Step Two: Create the neighborhood around $x_0$.**

A common way to choose the neighborhood size is to choose is large enough to capture proportion $\alpha$ of the data. This parameter $\alpha$ is often called the span. A typical choice is $\alpha \approx 0.5$.

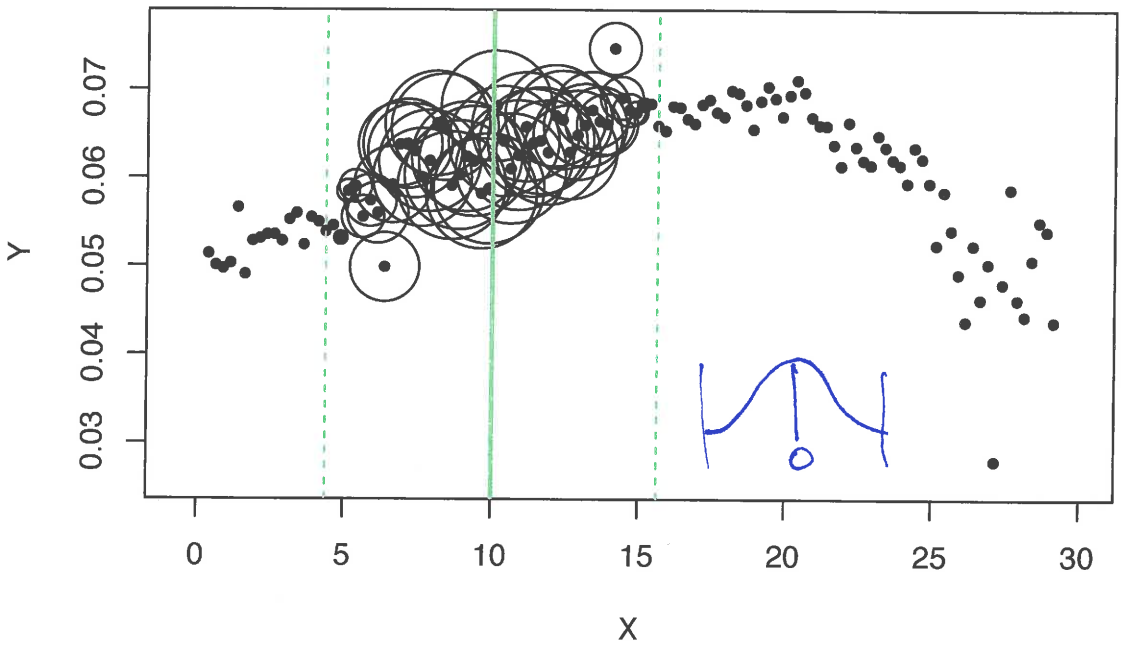





**Step Three: Weight the data in the neighborhood.**

Values of $x$ which are close $x_0$ will receive a larger weight than those far from $x_0$. Denote by $w_i$ the weight placed on observation $i$. The default choice is the tri-cube weight function:

$$w_i = \begin{cases} \left(1 - \left|\frac{x_i - x_0}{\text{max dist}}\right|^3\right)^3, & \text{if } x_i \text{ in the neighborhood of } x_0 \\ 0, & \text{if } x_i \text{ is not in neighborhood of } x_0 \end{cases}$$

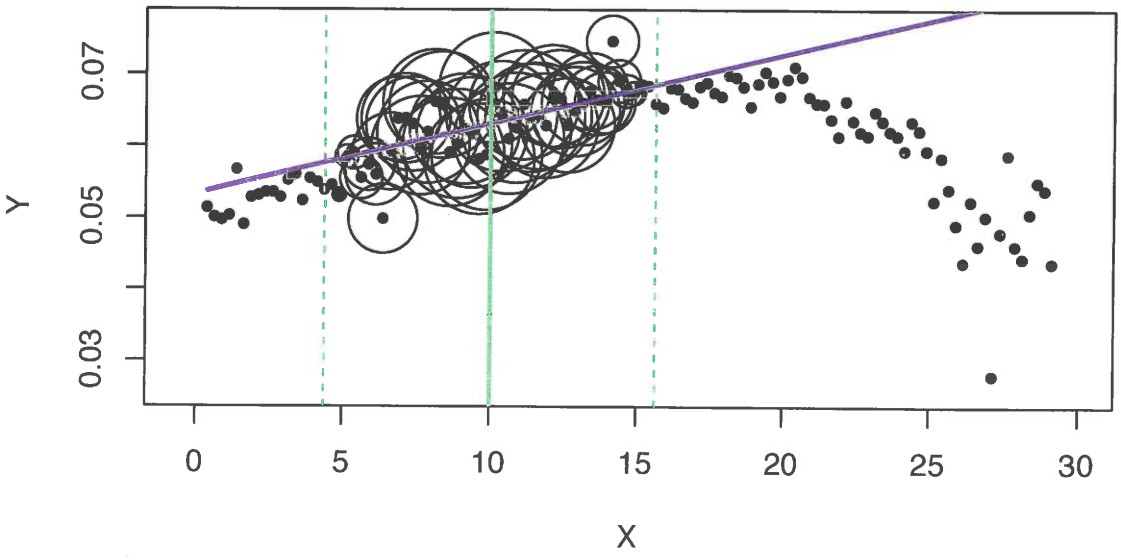





**Step Four: Fit the local regression line.**

This is done by finding $\beta_0$ and $\beta_1$ to minimize the weighted sum of squares

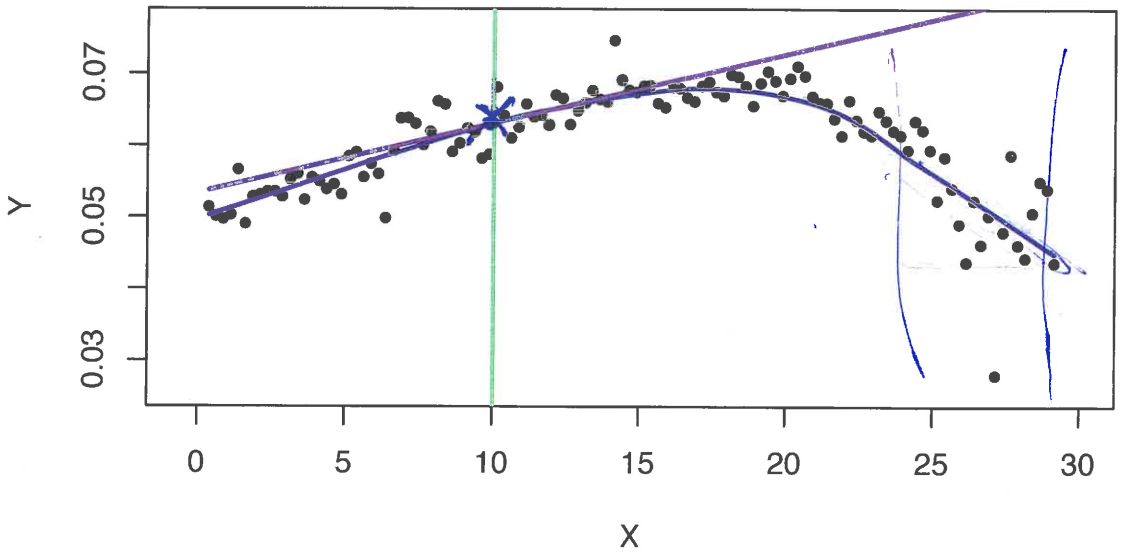
$$\sum_{i=1}^{n} w_i \left(y_i - (\beta_0 + \beta_1 x_i)\right)^2$$



**Step Five: Estimate $f(x_0)$.**

This is done using the fitted regression line to estimate the regression function at $x_0$:

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$$





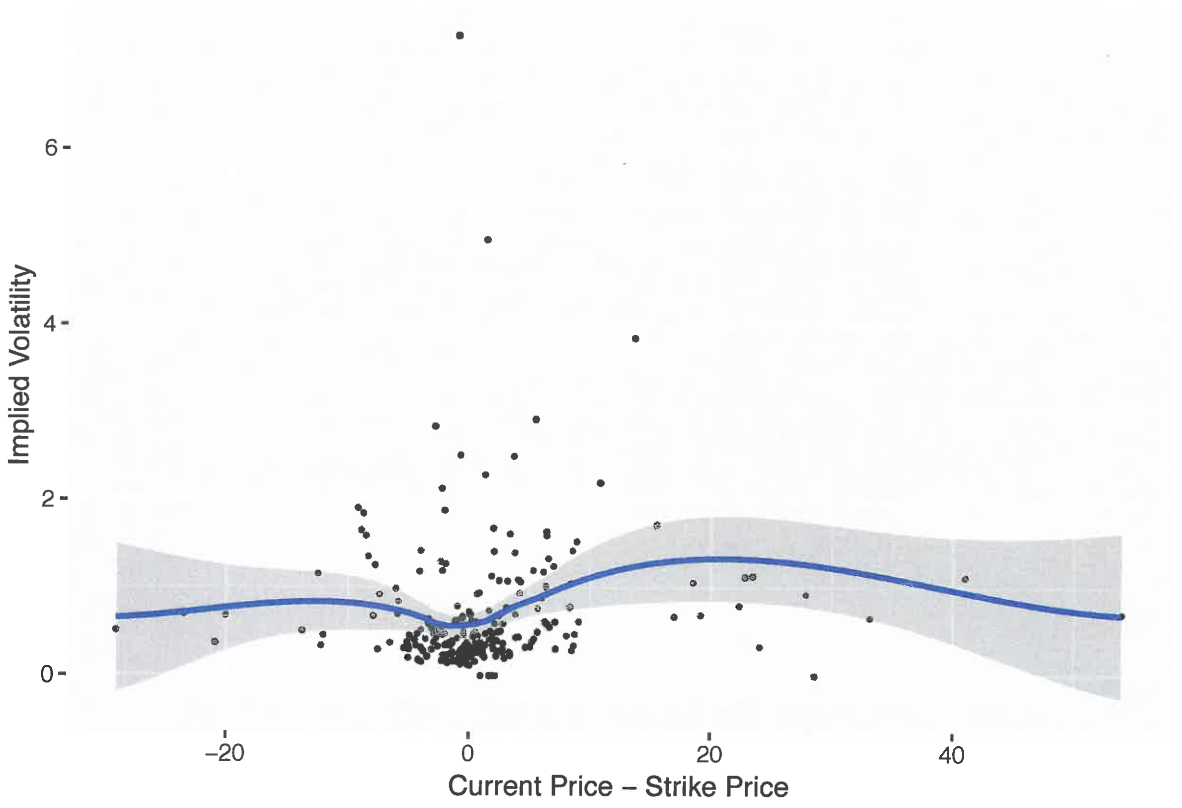

This is implemented in R using `loess()`:

```
> holdlo = loess(implvol ~ inmoney,
+                data=optionsdata20, degree=1)
```

Setting `degree = 1` is necessary in order to get local linear models. You can use the default `degree = 2` to get local quadratic fits, but this is usually unnecessary.

`ggplot` makes it easy to place the local linear regression fit onto the plot. Note how `degree=1` is passed on to `loess()`:

```
> ggplot(optionsdata20, aes(x=inmoney,y=implvol)) +
+   geom_point(size=0.5) + geom_smooth(method="loess",
+             method.args=list(degree=1)) +
+   labs(x="Current Price - Strike Price",
+        y="Implied Volatility")
```






## Smoothing parameter selection in Regression

Just like the choice of the bandwidth in kernel density estimation, the smoothing parameter in nonparametric regression controls how "smooth" the resulting estimate is: A smaller value leads to a "rougher" estimate, a larger value gives a "smoother" estimate.

The argument `span` to `loess()` controls the span, as defined above. The default value is 0.75.

**Exercise:** Try refitting the model above with smaller and larger values of `span`.

Automated approaches to smoothing parameter selection do exist. Different ideas are available, but a very popular approach is based on cross validation.

The motivation behind cross validation is that we seek a regression function that makes accurate predictions for **new** observations, not for those that are used to fit the model. A regression function that makes accurate predictions only for training data is said to be overfit. Overfitting is a serious problem, and is a leading drawback to the use of flexible nonparametric and machine learning methods.



**Exercise:** Discuss how nonparametric regression, and the choice of the smoothing parameter, leads to increased risk of overfitting. Does choosing the smoothing parameter to minimize residual sum of squares seem like a good idea?

As choose smoothing parameter smaller, the residual sum of squares $\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$ will decrease to 0. The model is capable of "perfectly" fitting to the training set, which would be extreme overfitting.

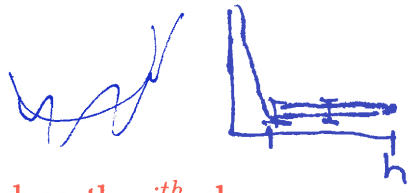
The residual sum of squares in this case is

$$\text{RSS} = \sum_{i=1}^{n} \left(Y_i - \hat{f}_h(x_i)\right)^2$$

where $\hat{f}_h(x_i)$ is the estimate of the regression function $f()$ when smoothing parameter $h$ is used.

Instead of choosing $h$ to minimize RSS, we minimize the leave-one-out cross validation score:

$$\text{LOOCV} = \sum_{i=1}^{n} \left(Y_i - \hat{f}_h^{(-i)}(x_i)\right)^2$$




where $f_h^{(-i)}(x_i)$ is the estimate of the regression function when the $i^{th}$ observation is excluded from the training set.

The idea here is that, by leaving out observation $i$, we are treating it as if it were a "new" observation, and testing to see how well this choice of $h$ does in predicting the response.

$$\left(Y_i - \hat{f}_h(x_i)\right)^2 < \left(Y_i - \hat{f}_h^{(-i)}(x_i)\right)^2 \quad \text{for all } i$$



There is another R function `loess.as()`, which is part of the package `fANCOVA`, which has built in selection of the span.

The syntax is

```
> holdlo2 = loess.as(optionsdata20$inmoney,
+         optionsdata20$implvol,criterion = "gcv")
```

With this function the default is `degree = 1`. Setting `criterion="gcv"` uses generalized cross validation, which is an approximation to leave-one-out cross validation.

The optimal span can be found via

```
> holdlo2$pars$span
[1] 0.7038925
```